

交互式曲线绘制软件 - 代码解释文档

本文档旨在对本项目核心代码进行逐行解释，特别是前端原生 Canvas 的底层绘制算法、事件处理，以及后端核心接口。

1. 项目结构说明

- 后端 (**backend/**)
 - **controller/**: RESTful 接口层，处理 HTTP 请求。
 - **service/**: 业务逻辑层，实现曲线数据处理。
 - **repository/**: 数据库访问层，基于 Spring Data JPA。
 - **entity/**: 数据表映射实体类。
- 前端 (**frontend/**)
 - **components/**: Vue UI 组件。**CanvasDrawer.vue** 承担核心绘制。
 - **utils/**: 核心数学算法（插值计算、几何碰撞检测）。
 - **stores/**: Pinia 状态管理。

2. 前端核心逻辑解释

本项目强调“底层实现、不调包”，因此最关键的部分在于数学插值算法和 Canvas 渲染逻辑。

2.1 曲线插值算法 (**utils/interpolation.ts**)

采用 Catmull-Rom 样条插值算法计算曲线点。相比标准贝塞尔曲线，Catmull-Rom 可以确保生成的曲线经过每一个控制点，更适合当前交互式绘制的场景。

```
export function getCatmullRomCurve(  
  points: Point[],  
  segments: number = 20,  
  closeCurve: boolean = false,  
  tension: number = 0.5  
): Point[] {  
  // 如果点少于2个，无法构成曲线，直接返回  
  if (points.length < 2) return [...points];  
  
  const result: Point[] = [];  
  const p = [...points]; // 拷贝控制点数组  
  
  if (closeCurve) {  
    // 闭合曲线：在头尾各追加点，形成环形闭包  
    p.unshift(points[points.length - 1]);  
    p.push(points[0]);  
    p.push(points[1]);  
  } else {
```

```

// 开放曲线：复制首尾点，保证端点处的张力计算正常
p.unshift(points[0]);
p.push(points[points.length - 1]);
}

// 遍历所有相邻的四点组 (p0, p1, p2, p3)
for (let i = 1; i < p.length - 2; i++) {
  const p0 = p[i - 1];
  const p1 = p[i];
  const p2 = p[i + 1];
  const p3 = p[i + 2];

  // 在 p1 和 p2 之间细分 segments 个插值点
  for (let t = 0; t <= segments; t++) {
    // 避免终点处的点重复加入数组
    if (t === segments && i < p.length - 3) continue;

    const t1 = t / segments; // 归一化进度  $t1 \in [0, 1]$ 
    const t2 = t1 * t1;      // t1 的平方
    const t3 = t2 * t1;      // t1 的立方

    // 计算 X 坐标的插值：基于张力参数(tension)混合四点坐标
    const x = 0.5 * (
      (2 * p1.x) +
      (-p0.x + p2.x) * tension * t1 +
      (2 * p0.x - 5 * p1.x + 4 * p2.x - p3.x) * tension * t2 +
      (-p0.x + 3 * p1.x - 3 * p2.x + p3.x) * tension * t3
    );

    // 计算 Y 坐标的插值
    const y = 0.5 * (
      (2 * p1.y) +
      (-p0.y + p2.y) * tension * t1 +
      (2 * p0.y - 5 * p1.y + 4 * p2.y - p3.y) * tension * t2 +
      (-p0.y + 3 * p1.y - 3 * p2.y + p3.y) * tension * t3
    );

    result.push({ x, y }); // 记录插值点
  }
}

return result; // 返回完整的细分点集
}

```

2.2 几何碰撞检测 (utils/geometry.ts)

为了判断用户鼠标是否点击到了已有顶点，需要手动实现碰撞检测算法。

```
/**
 * 欧几里得距离计算
 */
export function distance(p1: Point, p2: Point): number {
  const dx = p1.x - p2.x;
  const dy = p1.y - p2.y;
  return Math.sqrt(dx * dx + dy * dy);
}

/**
 * 碰撞检测：寻找第一个被鼠标点击到的顶点
 * 采用逆序遍历，优先响应视觉上位于最上层的顶点
 */
export function findCollidingVertex(point: Point, vertices: Point[],
radius: number = 8): number {
  for (let i = vertices.length - 1; i >= 0; i--) {
    if (distance(point, vertices[i]) <= radius) {
      return i; // 找到碰撞点，返回其索引
    }
  }
  return -1; // 未发生碰撞
}
```

2.3 Canvas 绘制与事件监听 (CanvasDrawer.vue)

该组件负责 Canvas 元素的生命周期，并通过 Vue `watch` 监听数据变化，触发重绘。

```
// 渲染函数
const render = () => {
  const canvas = canvasRef.value;
  if (!canvas) return;
  const ctx = canvas.getContext('2d');
  if (!ctx) return;

  // 清空画布，为重绘做准备
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  const vertices = [...curveStore.vertices];

  // 实时预览逻辑：将鼠标当前位置加入点集
  if (curveStore.mode === 'draw' && previewPoint.value &&
vertices.length > 0) {
    vertices.push(previewPoint.value);
  }

  // 绘制曲线本身
  if (vertices.length > 1) {
```

```
// 调用插值算法获取光滑曲线的点集
const curvePoints = getCatmullRomCurve(vertices, 20, false,
curveStore.settings.tension);

ctx.beginPath();
ctx.moveTo(curvePoints[0].x, curvePoints[0].y); // 移动到起点
for (let i = 1; i < curvePoints.length; i++) {
  ctx.lineTo(curvePoints[i].x, curvePoints[i].y); // 连线
}

// 设置曲线样式
ctx.strokeStyle = '#1e3a8a';
ctx.lineWidth = curveStore.settings.lineWidth;
ctx.lineCap = 'round';
ctx.lineJoin = 'round';
ctx.stroke(); // 实际执行绘制
}

// 遍历绘制所有顶点（带有选中状态高亮）
curveStore.vertices.forEach((v, index) => {
  ctx.beginPath();
  ctx.arc(v.x, v.y, curveStore.settings.vertexRadius, 0, Math.PI * 2);

  if (curveStore.selectedVertexIndex === index) {
    ctx.fillStyle = '#f97316'; // 选中状态：橙色
  } else {
    ctx.fillStyle = '#ef4444'; // 默认状态：红色
  }

  ctx.fill();
  ctx.stroke();
});
});

// 鼠标按下事件：判断新增顶点还是拖拽顶点
const handleMouseDown = (evt: MouseEvent) => {
  const pos = getMousePos(evt); // 获取画布内的相对坐标
  // 碰撞检测
  const clickedIndex = findCollidingVertex(pos, curveStore.vertices,
curveStore.settings.vertexRadius * 2);

  if (curveStore.mode === 'draw') {
    if (clickedIndex === -1) {
      curveStore.addVertex(pos); // 空白处点击，新增顶点
    } else {
      curveStore.selectedVertexIndex = clickedIndex; // 点击已有顶点，选中
    }
  } else if (curveStore.mode === 'select') {
    if (clickedIndex !== -1) {
      curveStore.selectedVertexIndex = clickedIndex;
      isDragging.value = true; // 开启拖拽状态
    }
  }
}
```

```
// 记录鼠标按下位置与顶点圆心的偏移量，保证拖拽平滑
dragOffset.value = {
  x: curveStore.vertices[clickedIndex].x - pos.x,
  y: curveStore.vertices[clickedIndex].y - pos.y
};
}
}

render(); // 触发重绘
};
```

3. 后端核心逻辑解释

后端提供简单的 REST API 用于曲线数据持久化。

3.1 曲线持久化服务 (CurveServiceImpl.java)

```
@Override
@Transactional
public CurveDTO createCurve(CurveDTO curveDTO) {
    Curve curve = new Curve();
    curve.setName(curveDTO.getName() != null ? curveDTO.getName() :
        "Unnamed Curve");

    // 遍历解析传入的顶点数据 DTO
    if (curveDTO.getVertices() != null) {
        for (int i = 0; i < curveDTO.getVertices().size(); i++) {
            VertexDTO vDto = curveDTO.getVertices().get(i);
            Vertex vertex = new Vertex();
            vertex.setX(vDto.getX());
            vertex.setY(vDto.getY());
            // 保证顶点顺序
            vertex.setOrderIndex(vDto.getOrderIndex() != null ?
                vDto.getOrderIndex() : i);
            curve.addVertex(vertex); // 添加关联关系
        }
    }

    // 使用 Spring Data JPA 保存级联数据
    Curve savedCurve = curveRepository.save(curve);
    return convertToDTO(savedCurve);
}
```

3.2 级联保存实体 (Curve.java)

```
@Entity
@Table(name = "curves")
public class Curve {
    // ...省略其他字段...

    // 一对多关系映射：删除曲线时自动删除关联的顶点
    @OneToMany(mappedBy = "curve", cascade = CascadeType.ALL,
        orphanRemoval = true)
    @OrderBy("orderIndex ASC") // 保证从数据库读取时顶点顺序正确
    private List<Vertex> vertices = new ArrayList<>();

    // 辅助方法：保证双向关联正确设置
    public void addVertex(Vertex vertex) {
        vertices.add(vertex);
        vertex.setCurve(this);
    }
}
```

4. 总结

通过上述源码分析可以看出，本项目在前端严格遵循了**不调包**的要求，手动实现了 Catmull-Rom 插值算法生成平滑曲线，并通过原生 Canvas API 完成了高质量的交互式绘制。后端采用主流的 SpringBoot 架构，完成了轻量可靠的曲线数据存储接口。